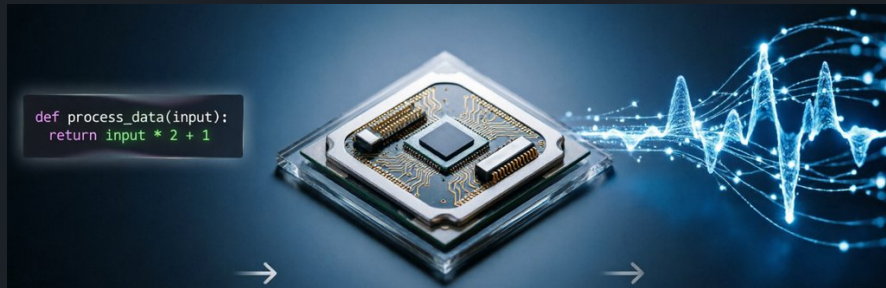


# ЛЕКЦИЯ №2

## Как компьютер выполняет программу *от кода до электричества*



Автор: Леонтьев Д.А.

# План лекции

---

## Часть 1 — Теория

- Что такое компьютер и программа
- Краткая история ЭВМ
- Уровни абстракции
- Архитектура фон Неймана
- Ключевые компоненты ПК
- Зачем программисту знать про железо
- Процесс выполнения программы

## Часть 2 — Демонстрация

- Написание простой программы
- Компиляция и поиск .exe
- Просмотр в HEX-редакторе
- Декомпиляция: от .exe обратно к коду
- Эксперименты с нагрузкой на CPU
- Эксперименты с памятью
- Сравнение Debug vs Release

ЧАСТЬ 1

# Теория

## РАЗДЕЛ 1

# Что такое компьютер?

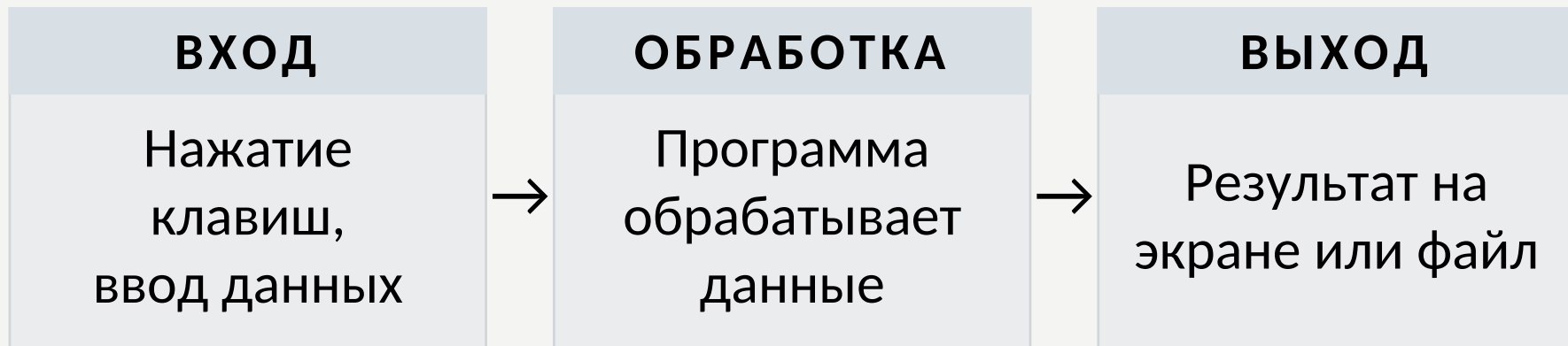
Обработка данных: вход, процесс, выход

# ЧТО ТАКОЕ КОМПЬЮТЕР

## Устройство для обработки данных

---

Компьютер - устройство для обработки информации по заданному алгоритму



*Принцип одинаков для любого устройства: калькулятора, игровой консоли, суперкомпьютера или смартфона.*

## ПРИМЕРЫ ИЗ ЖИЗНИ

# Вход → Обработка → Выход

---

### Калькулятор

**Вход:**  $2 + 2$

**Выход:** 4

### Игра

**Вход:** Нажатие клавиши

**Выход:** Движение героя

### Браузер

**Вход:** URL-адрес

**Выход:** Веб-страница

### Компилятор

**Вход:** Код на C#

**Выход:** .exe файл

РАЗДЕЛ 2

# История вычислений

От ENIAC до смартфонов и облаков

1940-Е ГОДЫ

# ENIAC — первый электронный компьютер

**Целая  
комната**

Размер

**27 тонн**

Вес

**150 кВт**

Потребление

**5 000 оп/сек**

Скорость

**Программирование:** перестановка  
физических проводов — вручную!

*Занимал около 167 м<sup>2</sup>, потреблял энергию как  
небольшой завод и требовал ручной  
перекоммутации для смены задачи.*





1950-60-Е ГОДЫ

## Первые языки программирования

Перфокарты заменили перестановку проводов. Появились первые языки высокого уровня.

**FORTRAN**

1957

Научные и инженерные вычисления. Первый коммерческий язык высокого уровня.

**COBOL**

1959

Деловые и финансовые операции. До сих пор используется в банках!

**Lisp**

1958

Символьные вычисления и ИИ. Заложил основы функционального программирования.

# Fortran 77 Examples

C Hello World in Fortran 77

C (lines must be 6 characters indented)

```
PROGRAM HELLOW
```

```
WRITE(UNIT=*, FMT=*) 'Hello World'
```

```
END
```

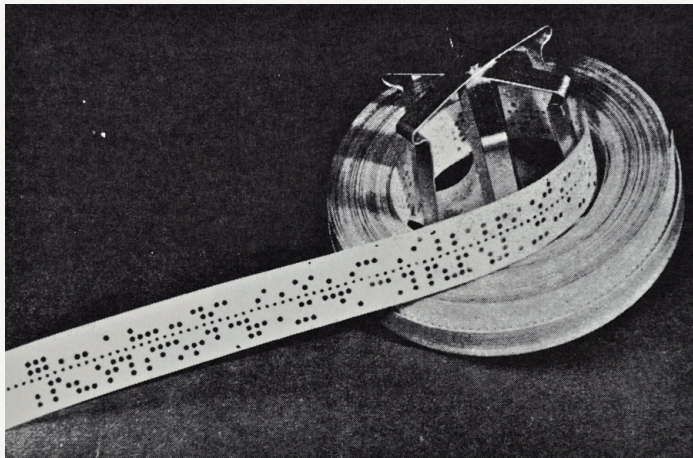
```
PROGRAM SQUARE
```

```
DO 15, I = 1,10
```

```
WRITE(*,*) I*I
```

```
15 CONTINUE
```

```
END
```



1970-2000-Е ГОДЫ

# Революция персональных компьютеров

---

| Период | Устройства                | Технологии          | Ключевое событие           |
|--------|---------------------------|---------------------|----------------------------|
| 1970-е | Apple II,<br>Commodore 64 | Intel 8080          | ПК на столе                |
| 1980-е | IBM PC, клоны             | Архитектура x86     | MS-DOS, рынок ПК           |
| 1990-е | Мультимедиа-ПК            | Windows, CD-ROM     | World Wide Web<br>(1991)   |
| 2000-е | Ноутбуки,<br>нетбуки      | Многоядерный<br>CPU | Широкополосный<br>интернет |

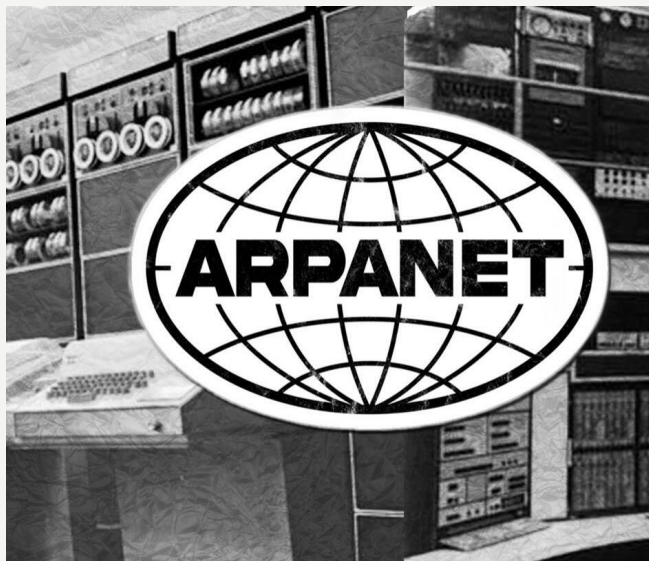
## 1970-е: Персональные компьютеры

- Apple II, Commodore 64
- Помещаются на столе



## 1980-2000-е: Революция ПК

- IBM PC, Windows, Intel x86
- Интернет, графический интерфейс



2010-Е — СЕГОДНЯ

## Эра мобильных устройств и облаков

---

### Смартфоны

- Мощнее суперкомпьютеров 1990-х
- App Store и Google Play
- GPS, камера, датчики — в кармане
- Постоянный доступ к интернету

### Облака и ИИ

- AWS, Azure, Google Cloud
- Вычисления без собственного сервера
- ChatGPT, LLM, генеративный ИИ
- Нейронные сети и машинное обучение

РАЗДЕЛ 3

# Уровни абстракции

От высокоуровневого кода до физических транзисторов

# ВСЕ УРОВНИ АБСТРАКЦИИ

## От кода до электричества

6

Высокоуровневый код (C#, Python, Java)

Компилятор / Интерпретатор ↓

5

Промежуточный код (IL, Bytecode)

JIT / Интерпретатор ↓

4

Машинный код (Ассемблер x86-64)

Декодер процессора ↓

3

Микрокоманды (Управляющие сигналы АЛУ)

Логические схемы ↓

2

Логические элементы (AND, OR, NOT, XOR, сумматоры)

Транзисторы ↓

1

Физический уровень (Транзисторы: ток есть / нет)



## УРОВЕНЬ 6 — ВЫСОКОУРОВНЕВЫЙ КОД

# C#, Python — то, что пишет программист

---

Высокоуровневый код близок к человеческому языку.

Компилятор или интерпретатор преобразует его в машинные инструкции.

```
// Код C# — сложение двух чисел
int a = 10;
int b = 20;
int sum = a + b;
Console.WriteLine(sum);
```

## УРОВЕНЬ 5 — ПРОМЕЖУТОЧНЫЙ КОД (IL / BYTECODE)

# Промежуточное представление для .NET

---

Компилятор C# переводит код в IL (Intermediate Language). Это уже не C#, но ещё не машинный код процессора.

```
// IL-код:  int sum = a + b;  
ldloc.0    // Загрузить 'a' в стек  
ldloc.1    // Загрузить 'b' в стек  
add        // Сложить два верхних значения  
stloc.2    // Сохранить результат в 'sum'
```

## УРОВЕНЬ 4 — МАШИННЫЙ КОД (АССЕМБЛЕР x86-64)

# Инструкции, которые понимает процессор

ЛТ-компилятор преобразует IL в инструкции архитектуры x86-64. Каждая инструкция кодируется числом и напрямую исполняется процессором.

```
// Ассемблер x86-64
```

```
MOV EAX, [rbp-4] ; a → EAX
```

```
ADD EAX, [rbp-8] ; EAX = EAX + b
```

```
MOV [rbp-12], EAX ; sum = EAX
```

## Внутри процессора: схемы и сигналы

### Уровень 3 — Микрокоманды

Декодер CPU разбивает ассемблерную инструкцию на микрооперации ( $\mu ops$ ).

Пример — `MOV EAX, [rbp-4]`:

- вычислить адрес `[rbp-4]`
- прочитать значение из памяти
- поместить в регистр `EAX`

### Уровень 2 — Логические элементы

Базовые схемы: AND, OR, NOT, XOR, сумматоры.

Из них строятся:

- регистры (хранение битов)
- АЛУ (арифметика)
- кэш-память
- управляющие схемы CPU

## УРОВЕНЬ 1 — ФИЗИЧЕСКИЙ

# Транзисторы: основа всего

---

Всё цифровое вычисление сводится к транзисторам — крошечным переключателям, которые пропускают или блокируют электрический ток.

**Ток есть → 1**

Транзистор открыт.  
Напряжение высокое.  
Логическая «1».

**Тока нет → 0**

Транзистор закрыт.  
Напряжение низкое.  
Логический «0».

Современный процессор содержит ~10 миллиардов транзисторов, каждый размером ~3–5 нм.

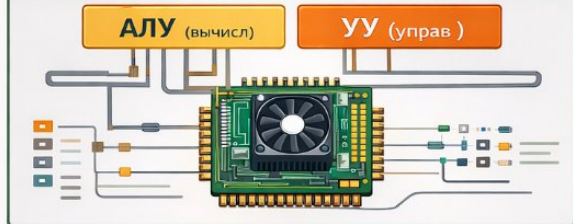
РАЗДЕЛ 4

# Архитектура фон Неймана

# АРХИТЕКТУРА ФОН НЕЙМАНА (1945 —)

Основа всех современных компьютеров

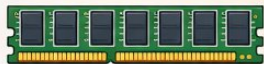
## ЦЕНТРАЛЬНЫЙ ПРОЦЕССОР (CPU)



Шина данных (Bus)

## ОПЕРАТИВНАЯ ПАМЯТЬ (RAM)

- Программы
- Данные



Шина данных (Bus)

## УСТРОЙСТВА ВВОДА-ВЫВОДА



# АРХИТЕКТУРА ФОН НЕЙМАНА (1945)

## ЦЕНТРАЛЬНЫЙ ПРОЦЕССОР (CPU)

АЛУ  
(Arithmetic  
Logic  
Unit)

РЕГИСТРЫ  
(быстрая  
память)  
  
EAX, EBX...

## УСТРОЙСТВО УПРАВЛЕНИЯ (УУ)

- Выборка инструкций
- Декодирование
- Управление выполнением

ШИНА ДАННЫХ  
(Bus: адрес, данные, управление)

## ОПЕРАТИВНАЯ ПАМЯТЬ (RAM)

| Адрес  | Содержимое           |
|--------|----------------------|
| 0x0000 | Инструкция: MOV...   |
| 0x0004 | Инструкция: ADD...   |
| 0x0008 | Данные: переменная a |
| 0x000C | Данные: переменная b |
| ...    | ...                  |

## УСТРОЙСТВА ВВОДА-ВЫВОДА (I/O)

ВВОД: Клавиатура, Мышь, Микрофон  
ВЫВОД: Монитор, Динамики, Принтер  
ХРАНИЕНИЕ: SSD, HDD, USB-флешка

РАЗДЕЛ 5

# Ключевые компоненты современного ПК

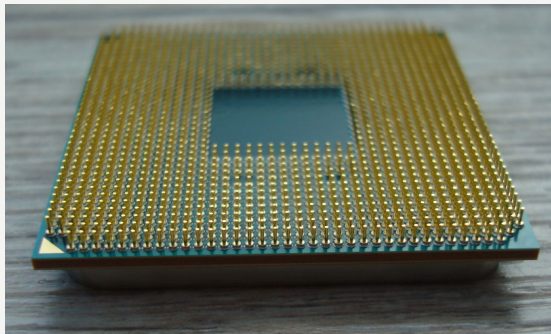


# ПРОЦЕССОР (CPU)

- **Ядра:** 2, 4, 8, 16... (многозадачность)
- **Частота:** 2-5 ГГц (операций в секунду)
- **Кэш:** L1 (64 КБ), L2 (512 КБ), L3 (8-32 МБ)
- **Примеры:** Intel Core i5, AMD Ryzen 5



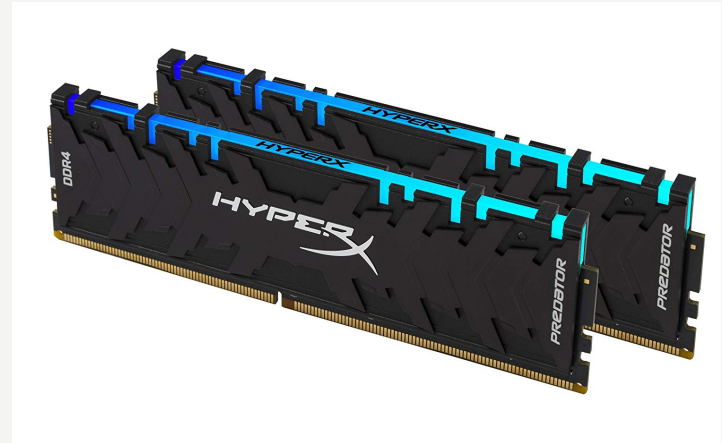
Процессоры от AMD



Процессоры от Intel

# ОПЕРАТИВНАЯ ПАМЯТЬ (RAM)

- **Тип:** DDR4, DDR5 (стандарты скорости)
- **Объём:** 4, 8, 16, 32+ ГБ
- **Скорость:** 2400-6000 МГц (МегаТрансферы/сек)
- **Энергозависимая!** (выключил ПК → всё стёрлось)



# НАКОПИТЕЛИ

## HDD (жёсткий диск):

- Механический (крутятся пластины)
- Медленный: ~100 МБ/сек чтение
- Дешёвый: ~2₽ за 1 ГБ



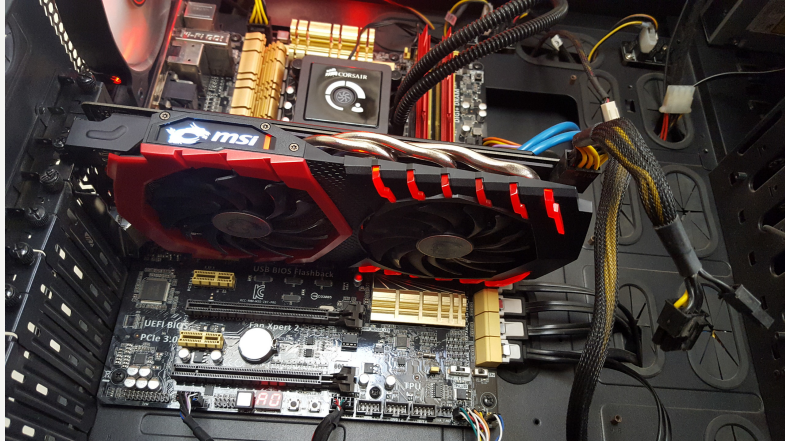
## SSD (твердотельный):

- Без движущихся частей (флеш-память)
- Быстрый: 500-7000 МБ/сек
- Дороже: ~5-10₽ за 1 ГБ



# ВИДЕОКАРТА (GPU) — опционально

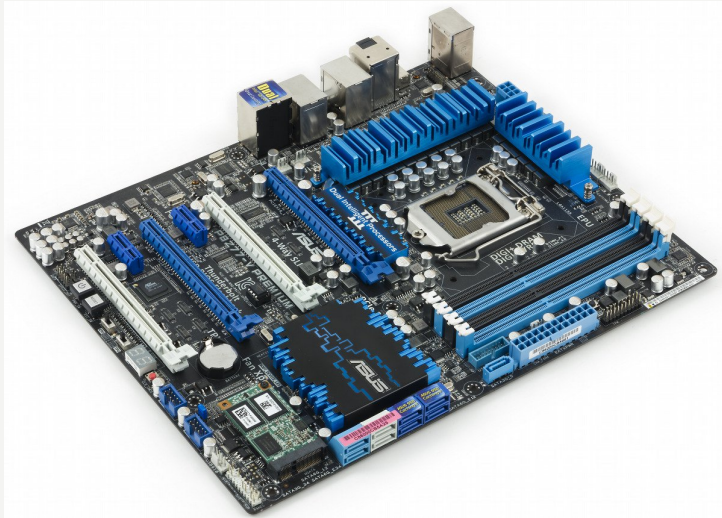
- Специализированный процессор для графики
- Тысячи простых ядер (vs десятки у CPU)
- Используется для игр, ML, рендеринга





# МАТЕРИНСКАЯ ПЛАТА

- "Скелет" ПК, соединяет все компоненты
- Разъёмы: CPU, RAM, GPU, SSD, питание
- Чипсет: управляет обменом данными



# БЛОК ПИТАНИЯ (PSU)

- Преобразует 220V → 12V, 5V, 3.3V
- Мощность: 300-1000 Вт



РАЗДЕЛ 6

# Зачем программисту знать про железо?

## ✗ МИФЫ:

- "Я пишу код, железо — не моё дело"
- "Компилятор сам всё оптимизирует"
- "Производительность не важна, железо дешёвое"
- "Это знание устарело, сейчас всё абстрагировано"



## **РЕАЛЬНОСТЬ:**

### **1. ПРОИЗВОДИТЕЛЬНОСТЬ И ОПТИМИЗАЦИЯ**

**Пример 1:** Программа обработки фото тормозит

**Причина:** загружаете весь 50 МБ файл в память, а у пользователя RAM всего 4 ГБ (половина занята Windows) → начинается swar на диск → всё замедляется в 100 раз

**Решение:** обрабатывать по кускам (streaming)

**Пример 2:** Игра лагает при 100+ объектах

**Причина:** создаёте новый объект каждый кадр →  
GC (сборщик мусора) задыхается → фризы

**Решение:** Object Pooling (переиспользование)

## 2. ОТЛАДКА "СТРАННЫХ" ОШИБОК

- "OutOfMemoryException" — кончилась память
- "StackOverflowException" — переполнен стек (обычно из-за бесконечной рекурсии)
- Программа тормозит → открыли Task Manager → видим: RAM 100%, CPU 5% → проблема в памяти!

### 3. ВЫБОР ПРАВИЛЬНЫХ ИНСТРУМЕНТОВ

- Многопоточность: имеет смысл, если CPU >1 ядра
- Базы данных: SSD vs HDD — разница в 50х
- Кэширование: понимание L1/L2/L3 помогает писать cache-friendly код (быстрее!)

## 4. СОБЕСЕДОВАНИЯ (middle/senior уровень)

### Типичные вопросы:

- "В чём разница между стеком и кучей?"
- "Почему массив быстрее связного списка?"  
(cache locality!)
- "Как работает виртуальная память?"

## **5. ОБЛАЧНЫЕ ВЫЧИСЛЕНИЯ И СТОИМОСТЬ**

**В облаке (AWS, Azure) вы платите за:**

- CPU (за час использования)
- RAM (за ГБ·час)
- Трафик (за ГБ переданных данных)

**Неоптимальный код = лишние деньги!**

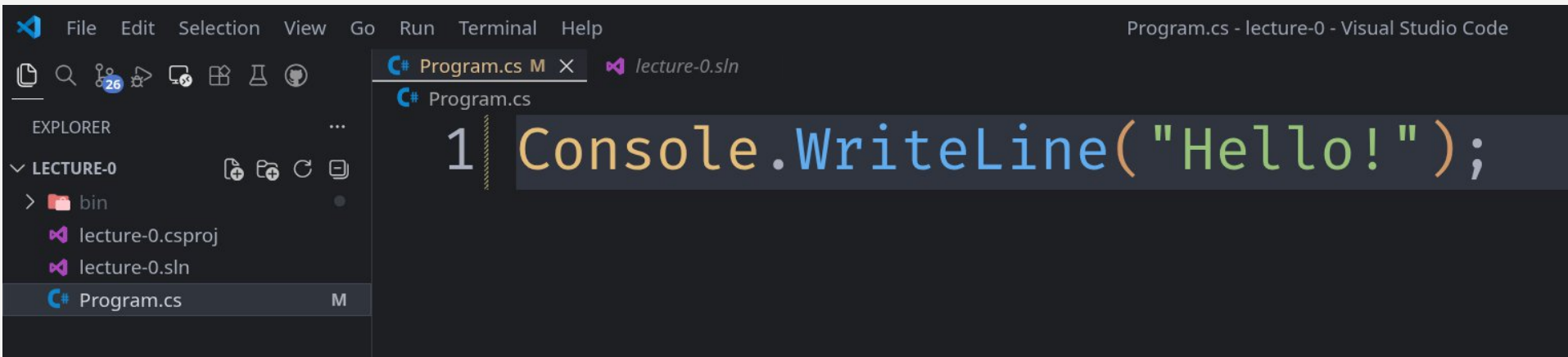
**Реальный кейс:** оптимизация запроса к БД сократила расходы на AWS с \$5000 до \$500/мес

РАЗДЕЛ 7

# Процесс выполнения программы

# ШАГ 1: НАПИСАНИЕ КОДА

Программист → текстовый редактор → Program.cs  
*Console.WriteLine("Hello!");*





## ШАГ 2: КОМПИЛЯЦИЯ

*Program.cs* → Компилятор C# (Roslyn) → IL-код

Что делает компилятор:

- Лексический анализ (разбивает код на токены)
- Синтаксический анализ (проверяет грамматику)
- Семантический анализ (проверяет смысл)
- Генерация IL-кода
- Оптимизация (в Release режиме)

Результат: **lecture-0.dll** (IL-код + метаданные)

 lecture-0.dll

```
>$ dotnet build
Восстановление завершено (0,4 с)
  lecture-0 net10.0 успешно выполнено (2,2 с) → bin/Debug/net10.0/lecture-0.dll
Сборка успешно выполнена через 2,8 с
```

## ШАГ 3: ЗАГРУЗКА В ПАМЯТЬ

Пользователь запускает *Program.exe* →

Операционная система (Windows):

- Выделяет виртуальное адресное пространство
- Загружает .NET Runtime (CLR)
- Загружает *Program.dll* в память
- Создаёт процесс (ID, приоритет, ресурсы)

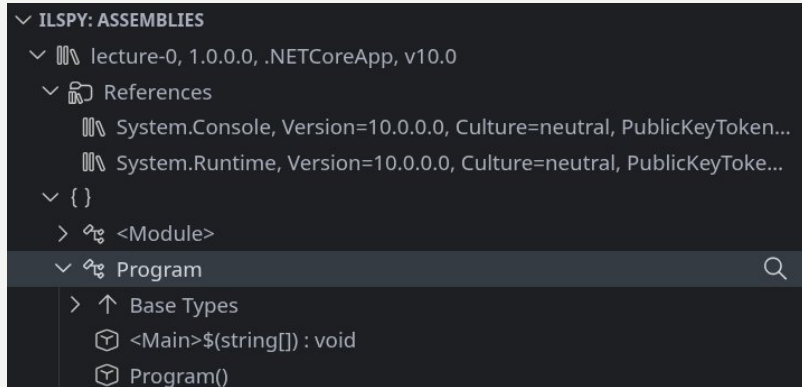
```
>$ ls
.rwxrwxrwx 98k denis 28 map 23:05 lecture-0
.rw-r--r-- 399 denis 28 map 13:44 {} lecture-0.deps.json
.rw-r--r-- 4,6k denis 28 map 23:05 lecture-0.dll
.rw-r--r-- 11k denis 28 map 23:05 lecture-0.pdb
.rw-r--r-- 259 denis 28 map 13:44 {} lecture-0.runtimeconfig.json
[denis@denis-Desktop:~/P/t/l/b/D/net10.0]-[23:08:00]-[G:main =]
>$ ./lecture-0
Hello!
```

## ШАГ 4: JIT-КОМПИЛЯЦИЯ

*IL-код → JIT-компилятор → машинный код*

### Почему не сразу в машинный код?

- IL-код платформонезависим (работает везде)
- JIT оптимизирует под конкретное железо:
  - Видит: CPU поддерживает AVX-512? → использует
  - Видит: кэш L3 большой? → адаптирует код



```
using System;
using System.Runtime.CompilerServices;

[CompilerGenerated]
internal class Program
{
    private static void <Main>$(string[] args)
    {
        Console.WriteLine("Hello!");
    }
}
```

## ШАГ 5: ИСПОЛНЕНИЕ (цикл процессора)

*Бесконечный цикл (пока программа не завершится):*

1. FETCH: Прочитать инструкцию из памяти
2. DECODE: Расшифровать (ADD, MOV, CALL...)
3. EXECUTE: Выполнить (АЛУ складывает числа)
4. WRITE: Записать результат
5. Increment PC (Program Counter)

→ Повторить

*Частота: 3 ГГц = 3 млрд циклов/сек*

## ШАГ 6: ЗАВЕРШЕНИЕ

- Программа вызывает `return/Environment.Exit()`
- ОС освобождает память процесса
- Закрывает файлы, сокет, ресурсы
- Процесс исчезает из Task Manager

ЧАСТЬ 2

# Живая демонстрация

# Что мы увидим

---

Сейчас мы увидим всё это В ДЕЙСТВИИ:

- Напишем программу на C#
- Скомпилируем её (увидим IL-код!)
- Посмотрим .exe в HEX-редакторе
- Декомпилируем обратно в C# (!)
- Запустим и увидим процесс в Task Manager
- Нагрузим CPU на 100%
- Съедем всю память (намеренно!)
- Сравним Debug vs Release

# Ключевые выводы

---

- Программа = инструкции для железа (байты)
- Компилятор переводит C# → IL → машинный код
- Процессор выполняет миллиарды операций/сек
- Память ограничена — нужно экономить  
Release в разы быстрее Debug



# Главное, что нужно запомнить

---

1. Компьютер — не магия. Это слои абстракции. Каждый уровень упрощает работу для уровня выше. Вы пишете  $sum = a + b$ , процессор получает `ADD EAX, EBX`, транзисторы переключают ток. Всё логично и предсказуемо.
2. Знание железа критично. Не для 'общего развития', а для реальной работы. Оптимизация, отладка, выбор инструментов, собеседования, экономия денег в облаках — везде нужно понимание того, что происходит 'под капотом'.

# Главное, что нужно запомнить

---

3. Практика показала теорию в действии. Вы увидели:

- Как код превращается в байты
- Как процессор нагружается до 100%
- Как память выделяется и освобождается
- Почему Release быстрее Debug

# **Спасибо за внимание**

Лекция №2 · Как компьютер выполняет программу ·  
от кода до электричества